© 2016. This manuscript version is made available under the CC-BY-NC-ND 4.0 license https://creativecommons.org/licenses/by-nc-nd/4.0/. The final version was published in Future Generation Computer Systems 68: 111–127 (2017), https://doi.org/10.1016/j.future.2016.08.016

Deriving Scientific Workflows from Algebraic Experiment Lines: A Practical Approach

Anderson Marinho[a,b], Daniel de Oliveira[c], Eduardo Ogasawara[d], Vitor Silva[a], Kary Ocaña[e], Leonardo Murta[c], Vanessa Braganholo[c§], Marta Mattoso[a]

[a]PESC/COPPE - UFRJ - Federal University of Rio de Janeiro, Rio de Janeiro, Brazil.
[b]PETROBRAS - Petróleo Brasileiro S.A., Rio de Janeiro, Brazil.
[c]UFF - Fluminense Federal University, Rio de Janeiro, Brazil.
[d]CEFET/RJ - Federal Center of Technological Education, Rio de Janeiro, Brazil.
[e]LNCC –Scientific Computing National Laboratory, Rio de Janeiro, Brazil

{andymarinho, silva, marta}@cos.ufrj.br, {danielcmo, leomurta, vanessa}@ic.uff.br, eogasawara@cefet-rj.br, karyann@lncc.br

Summary

The exploratory nature of a scientific computational experiment involves executing variations of the same workflow with different approaches, programs, and parameters. However, current approaches do not systematize the derivation process from the experiment definition to the concrete workflows and do not track the experiment provenance down to the workflow executions. Therefore, the composition, execution, and analysis for the entire experiment become a complex task. To address this issue, we propose the Algebraic Experiment Line (AEL). AEL uses a data-centric workflow algebra, which enriches the experiment representation by introducing a uniform data model and its corresponding operators. This representation and the AEL provenance model map concepts from the workflow execution data to the AEL derived workflows with their corresponding experiment abstract definitions. We show how AEL has improved the understanding of a real experiment in the bioinformatics area. By combining provenance data from the experiment and its corresponding executions, AEL provenance queries navigate from experiment concepts defined at high abstraction level to derived workflows and their execution data. It also shows a direct way of querying results from different trials involving activity variations and optionalities, only present at the experiment level of abstraction.

Keywords

Scientific Workflows, Software Product Line, Workflow Algebra, Workflow Derivation.

[§]Corresponding author

1. Introduction

Scientific workflows are used as an abstraction that structures the steps of a computational scientific experiment. They may be represented as a graph of activities, in which nodes correspond to data processing activities and edges represent the dataflow dependency between them [1]. Scientific workflows enable an organized composition of programs as a sequence of activities aiming at a specific result. Scientific workflow management systems (SWfMS) are becoming an indispensable tool in several scientific domains. For example, in the medical imaging area, scientific workflows are deployed on grids for addressing the computing and storage needs arising from the manipulation of large fragmented medical data sets on wide area networks [2]. Climatologists use scientific workflows for visual exploration of climate data [3]. Other examples can be found in domains such as physics [4], ecology [5], chemistry [6], astronomy [7], bioinformatics [8]–[10], and oil & gas [11].

SWfMS have been developed to support scientists in the process of composing, executing, and analyzing the results of a scientific workflow execution. Thanks to these systems, scientists do not need to perform tedious tasks as activities dispatching or data transferring from one program to another [12]. SWfMS manage the workflow orchestration, including its parallel execution. Furthermore, SWfMS improve knowledge sharing, reuse, repeatability, and data provenance capture of workflows [13].

The exploratory nature of a scientific computational experiment [14] involves not only executing variations of the same workflow using several data combinations but also changing workflow activities to explore different methodologies, approaches, or tools. To follow a scientific hypothesis, the scientist plans several workflow execution trials (*i.e.,* the different workflow executions that scientists experiment using different assumptions). We define the computational scientific experiment [15] as the set of all workflow executions that correspond to the planning and computational evaluation of a scientific hypothesis. From now on, we call it simply *experiment*.

Despite the highly effective SWfMS support for one workflow execution and analysis, they still lack a higher level support for the entire experiment composition, execution and analysis. The scientific experiment life cycle is not managed as a whole relating all workflow trials. Without computational support, the scientific hypothesis definition and the several workflow execution trials are kept only in the mind of the scientists. There is no representation or supporting system to integrate these trials within the same analytical environment. As a consequence, scientists may lose track of the trials, leading to rework and difficulties in hypothesis evaluation.

Existing SWfMS [16]–[22] represent a single workflow at a time [23], *i.e.,* only one of the possibilities of a workflow composition for an experiment. The importance of composing scientific experiments at abstraction levels higher than the ones provided by existing SWfMS has been reckoned before [3], [24]–[32]. However, these approaches fail at providing a model or a supporting system to integrate the resulting data from the execution of variations that an experiment can present within the same analytical environment. They do not formalize these variations in a way that allows for tracking and querying the workflows generated for the same experiment, classifying the origin of each derived workflow to its pre-defined variation.

The problem of representing scientific experiments in different levels of abstraction becomes particularly apparent when scientists attempt to analyze the results produced by different executions of scientific workflows as part of the same experiment. In this scenario, there are several missing links for connecting prospective and retrospective provenance data [33], particularly in the occurrence of distribution of concurrent activity executions in HPC environments. This prevents scientists to answer queries such as “which concrete workflows use program P1 and which use program P2 for the same abstract workflow activity?” or “which workflow related to the same experiment executed faster?” or even “which activities present variations for the same experiment?”. These queries are related to the experiment as a whole opposed to a single isolated workflow execution. Therefore, there is a need of a higher abstraction level that integrates the experiment information, in such a way that it can also be queried with the corresponding derivations and executions. This need led us to the following research question: “How can scientists answer queries about the experiment as a whole, transcending the concrete and abstract levels that are used to model a workflow?”. Our goal is to allow for scientists to describe their experiment in a high abstraction level, guaranteeing that provenance is tracked at all levels, from the higher conception

level until the execution of the (possibly several) workflows that represent the experiment. Thus, our approach allows for scientists to answer the former questions. This is a new level of experiment management enabling users to have a broader view of the experiment as whole. We call it **experiment level**.

In this paper, we address the problem of supporting an experiment with all its derived abstraction levels by proposing the algebraic experiment line approach (AEL). AEL extends and enhances the semantic support of the previously proposed concept of the experiment line [34]. AEL contributes by introducing a uniform data model that adds semantics to the derived workflows. These semantics represent what and how the data is consumed and produced in the derived workflows. Due to that, even in the abstract level, the experiment line is capable of verifying compatibility between workflow activities by comparing their associated schema. AEL builds on a workflow algebra [35], referred here as the SciWfA algebra. Since SciWfA algebra is applied in all abstraction levels supported by AEL, the algebraic representation allows for querying workflow execution data with their corresponding abstract experiment definition. The workflows derived from the AEL can be used by a SWfMS compatible to the SciWfA algebra (such as SciCumulus [17]) and benefit from algebraic transformations to get the corresponding algebraic operators to do automatic optimizations in the workflow execution plan.

We formally define the algebraic concepts related to the AEL and their relationships and constraints to the SciWfA algebra. Additionally, we specify and implement a tool named ExpLine for supporting the creation of experiment lines and composition of workflows in different levels of abstraction. As a case study, we use a phylogenetic analysis experiment executed under SciCumulus [17], a cloud workflow engine used as a target SWfMS. Results show that queries combining data from the experiment line and the corresponding executions improve the data analysis and understanding of the experiment. It also shows a direct way of querying results from different trials involving activity variations, which are defined at the experiment line level of abstraction.

Scientists can benefit from a series of advantages by using the algebraic experiment line approach to represent the experiments at a higher level than an abstract workflow. The higher abstract model of AEL (I) relates the information of different workflows created during the experiment life cycle, representing in a (II) unique and integrated data model the existing workflows. Additionally, this promotes the experiment (III) reuse and (IV) reproducibility, since it formalizes the workflows common characteristics and their variations.

In summary, the contributions of this work are as follows:

- A well-defined and formalized workflow derivation process that now works at three abstraction levels: the experiment level; the abstract workflow level; and the concrete workflow level.
- A provenance approach that tracks the information generated in the AEL enabling the creation of provenance queries that correlates experiment line concepts defined at high abstraction level with derived workflows and their execution data.
- Incorporation of the SciWfA algebra into the experiment line approach to make both the data description and the operation explicit in the experiment abstraction level, thus improving consistency in the model.
- Specification and implementation of a prototype named ExpLine that supports the new functionalities present in AEL.
- A case study that evaluates the AEL approach on managing several trials of an experiment in the phylogenomic area.

The remaining of this paper is organized as follows. Section 2 formally presents the SciWfA workflow algebra. Section 3 presents a real astronomy data analysis experiment that is consistently used in the rest of the paper as our running example and to demonstrate our approach. Section 4 discusses related work. Section 5 presents the AEL approach and specifies the architecture of the ExpLine tool, which supports the creation and management of algebraic experiment lines. Section 6 presents the evaluation study of the AEL implementation with the phylogenomic analysis experiment. Finally, Section 7 concludes the paper.

2. SciWfA Workflow Algebra

This section presents a workflow formalism that encompasses both a data model, a dataflow model, and the corresponding operators using the SciWfA workflow algebra [35]. This algebra adopts relations as the basic data operands for the algebraic operators corresponding to data consumption and production between workflow activities. Assuming this data model, the scientific workflow is expressed as follows.

2.1. Data Model

Workflow input and output data as well as the combination of data consumed and produced by activities correspond to relations in the database relational model [36, p. 2]. In this context, during the execution, workflow activities consume and produce tuples that follow schema relations, which are defined using a set of primitive data (integer, real, alphanumeric, date, *etc*.) or references to files. Thus, each relation R is a set of tuples (*i.e.*, $\{r_1, ..., r_n\}$, considering n as the cardinality of R) that follows a schema $\mathcal{R}$, with its respective attribute types.

Each relation R presents the following basic properties: (i) it is composed of one or more attributes; (ii) each attribute has a unique name in the relation, a specific type and associated constraints; (iii) it has a set of attributes that together form a key (that guarantees uniqueness). An example of schema is presented below:

$$\mathcal{R} = (Attr1: Integer, Attr2: Char, Attr3: Boolean)$$

Definition 1. (SCIENTIFIC WORKFLOW) A scientific workflow w is a Directed Acyclic Graph (DAG) defined by the tuple $<Nodes, Edges>$. *Nodes* in w correspond to the set of workflow activities (*i.e.*, the set $A = \{a_1, .., a_n\}$, considering that A has n activities), whereas *Edges* correspond to the set of data dependency ($Dep(a_i, a_j)$) that exists between each pair (a_i, a_j) of activities in A, $i \neq j$. An activity a_i is a quadruple $<\{R_1,..., R_m\}, \mathcal{I}, P, op>$, which states that a_i consumes a set of relations $\{R_1,..., R_m\}$ with schemas respectively compatible with $\{\mathcal{R}_1,..., \mathcal{R}_m\}$ and produces an output relation with schema compatible with $\mathcal{I}$. Additionally, a_i is associated with an algebraic operator op. This means that a_i can only be invoked in association with that particular algebraic operator op. P is a program that is executed when a_i is invoked. In addition, $Input(a)$ defines the input relations consumed by activity a. Similarly, $Output(a)$ defines the output relation produced by a. This way, given two activities a_i and a_j, the dependency between them is defined as $Dep(a_i, a_j) \leftrightarrow \exists R_k \in Input(a_j) \mid R_k = Output(a_i)$.

2.2. Algebraic Operators

Algebraic operators are associated to workflow activities according to how activities consume and produce tuples. Operators add semantics to data consumption and production along workflow activities. There are five operators in SciWfA algebra: Map, Filter, SplitMap, Reduce, and MRQuery [35]. All operators are unary (*i.e.*, consume a single relation), with the exception of MRQuery, which is an n-ary operator (*i.e.*, consumes n relations). These algebraic operators are detailed as follows.

Definition 2. (MAP) *Map* is represented as $T \leftarrow Map(a, R)$. Map produces a single tuple t in the output relation T for each tuple r consumed by activity a from the input relation R. Considering n as the cardinality of the input relation R, *Map* executes activity a n times. *Map* is a one tuple consumption to one tuple production operation.

Definition 3. (FILTER) *Filter* is represented as $T \leftarrow Filter(a, R)$. It selects a tuple r from the input relation R to be copied to the output relation T, as long as r complies with a criterion defined in activity a. In this case, the output relation, which follows the schema T, is a subset of the input relation R, which follows the schema $\mathcal{R}$. Considering that the cardinality of the input relation R is n tuples, this operator executes activity a n times. The operator produces m selected tuples in the output relation T, such that $m \leq n$. The selection criterion is internal to the program represented by activity a.

Definition 4. (SPLITMAP) *SplitMap* is represented as $T \leftarrow SplitMap(a, R)$. It produces a set of tuples in the output relation T for each tuple from the input relation R consumed by activity a. The splitting

criterion is internal to the program represented by activity *a*. *SplitMap* is a one tuple consumption to *n* tuple production operation.

Definition 5. (REDUCE) *Reduce* is represented as $T \leftarrow Reduce(a, G_a, R)$. It produces a single tuple *t* in the output relation *T* from each subset of tuples in the input relation *R*. The tuples from *R* are grouped by a subset of *q* attributes from the relation *R*, which can be represented as $G_a=\{attr_1, ..., attr_q\}$. The set G_a is used to establish the criteria for horizontal partitioning over *R*. Summarizing, the Reduce operator executes the activity *a*, consuming each partition at a time and producing an aggregated tuple *t* for each partition, similarly to the MapReduce model [37].

Definition 6. (MRQUERY) *MRQuery* is represented as $T \leftarrow MRQuery(a, \{R_1,..., R_o\})$. It invokes activity *a* to consume tuples from a set of relations $\{R_1,..., R_o\}$ in order to produce a single output relation *T*. It is similar to the join operator in the relational algebra, but using more than two relations. Notice that the join criterion is internal to the program represented by activity *a*.

In Figure 1 of Section 3, these operators are illustrated in detail when instantiated in an astronomy experiment workflow.

2.3. Workflow Validation

Definition 7. (VALID SCIENTIFIC WORKFLOW) A minimal set of conditions for a scientific workflow *w* to be valid is:

- Each activity must be associated to an algebraic operator;
- The activity signature (*i.e.*, input and output relation cardinalities, the relation schema characteristics) must comply with its associated algebraic operator;
- The input and output relations of an activity must comply with their associated schemas; and
- The activities and their dependency relationships must comply with the DAG constraints.

3. Running Example: Astronomy Data Analysis Experiment

To illustrate a real problem of modeling several scientific workflows that represent different trials in the context of the same experiment, let us consider the Montage astronomy experiment [38] that creates image mosaics to perform data analysis, which has been modeled in SciWfA by Silva *et al.* [39]. We use Montage because it has been used as a benchmark in the analysis of scientific workflow techniques. We consider the Montage workflow [39] as the basis of an experiment that uses a series of tools for assembling astronomical images into custom mosaics suitable for large scale processing of the sky. It is possible to build mosaics in Flexible Image Transport System (FITS) format [40], according to common astronomy coordinate system, arbitrary image sizes and rotations, and all World Coordinate System (WCS) map projections.

In this example, we adapted the Montage workflow [39] so it could be expressed as a generic astronomy data analysis experiment, named MontageExp. MontageExp can be implemented by using either Montage or another similar astronomy image mosaic composition toolkit. Therefore, the workflow activities from MontageExp were redefined in a higher level of abstraction to ease the explanations. The workflow is composed of five activities as shown in Figure 1.a.

The first activity (List Images) transfers image files from an external astronomy repository and extracts every astronomy reference files from each image (which can be more than 20 types of domain data). Each image file corresponds to a mosaic creation. The second activity (Projection) computes the projection of these astronomy-positioning references into a specific plane. Here, we can find the first challenge. There are several coordinate systems that can be used to make the projection and each of them suits specific projection transformations. In our experiment, we consider the Cartesian, Spherical, and Celestial coordinate systems. The third activity (Calculate Overlaps) analyzes an image metadata table to determine which images overlap on the sky. In this astronomy experiment we selected two different algorithms to calculate overlaps. The first aims at generating better quality (with data correction) mosaics while the second focuses in performance (without data correction). The fourth activity (Calculate Correction) depends on the chosen algorithm for the previous activity. This fourth activity is only executed if the

workflow composition chooses the improved quality algorithm. This means that this activity can be suppressed if the higher quality data is not generated. Finally, the fifth activity (Create Mosaic) generates the mosaic joining all the images. As the previous activity (Calculate Correction), this activity depends on the image correction being previously processed. Therefore, two algorithms can be used to generate, respectively, a corrected or uncorrected mosaic. Uncorrected mosaics do not have color corrections, but are frequently used since they are faster to obtain and allow for scientists to have a short preview of what they are doing.

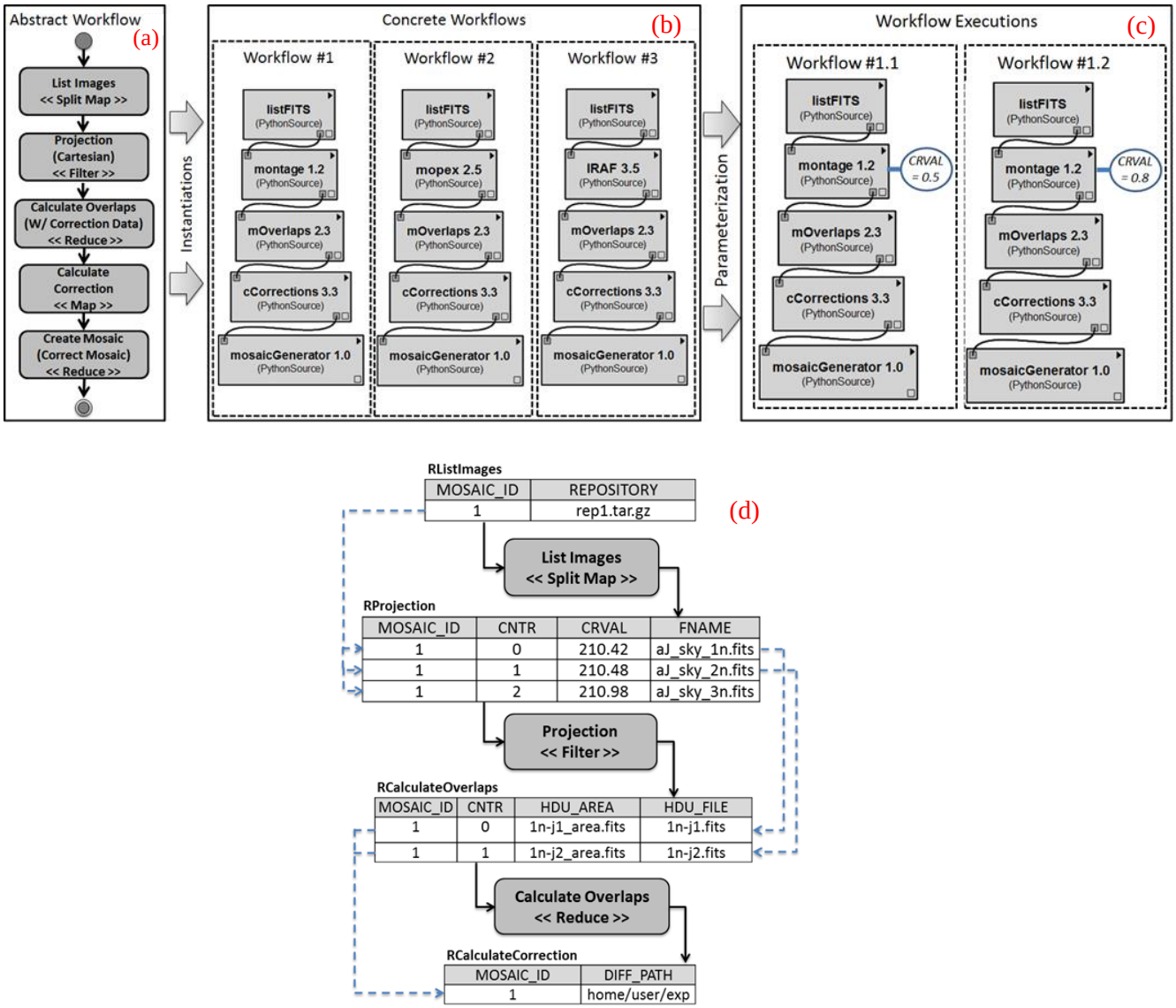


Figure 1. Astronomy data analysis workflow adapted from Silva *et al.* [39] represented at different abstraction levels (a, b, c) and the first three activities in detail with their relationships to the input and output relations (d)

Summarizing, when analyzing the variabilities and restrictions of this experiment, we find six different ways to create image mosaics. We have three coordinate systems to project the images and two options to create mosaics (with or without corrections) resulting in six distinct possible workflows to manage for a single experiment. It is worth mentioning that this experiment was chosen due to didactic reasons. Complex experiments with several variabilities and optionalities can have dozens of distinct possibilities of workflows.

3.1. Algebraic representation of the astronomy experiment

Figure 1.d illustrates in detail the SciWfA algebraic specification for the first three workflow activities of the astronomy experiment. Besides the algebraic operations from Figure 1.a, it explicitly shows the input

and the output relations for these three workflow activities. The input and output relations are represented as tables and the content of these tables are tuples, which represent possibilities of workflow data executions. For instance, the workflow activity List Images is defined as a Split Map operation and its input and output relations are named, respectively, as RListImages and RProjection. On the other hand, RProjection also acts as the input relation for the workflow activity Projection since this is dependent from the workflow activity List Images. This dependency is represented by a solid line arrow connecting the relation to the workflow activities.

Also, in Figure 1.d, the dotted line arrows represent the relation tuples transformations along the workflow. These tuple transformations occur according to the algebra operation specified for each activity. For example, the workflow activity List Images, characterized as a Split Map operation, takes one tuple from the input relation RListImages (which contains a tar.gz file with the sky images) and generates three tuples (the tree sky images uncompressed from tar.gz file) in the output relation RProjection. The provenance information defined in this fine grain level of detail is captured by the SciWfA algebra, which allows for scientists to improve reasoning of the workflow execution analysis. From that, it is possible to visualize that one of the sky images (aJ_sky_3n.fits) was discarded in the Projection activity. A coarse grain provenance support, which only maps the dataflow at relation level, could let scientists to have a wrong idea about the workflow execution. Scientists could conclude that all of the sky images included in the tar.gz were used to create the mosaic.

3.2. Supporting Different Levels of Abstractions: Limitations and Open Issues

Abstraction levels provide a separation of the experiment conceptual information from its implementation, which, according to the environment where the experiment is executed, can show several differences. Let us clarify this problem by using the astronomy experiment. In Figure 1, the astronomy data analysis workflow is represented in three different abstraction levels. Figure 1.a shows the abstract workflow of the astronomy experiment according to an agnostic notation and its implementations in VisTrails notation [16] (Figure 1.b) to represent concrete executable workflows and some examples of workflow executions with specific parameters (Figure 1.c). Although this Figure presents several alternative concrete workflows using the VisTrails notation, the discussion presented here transcends a particular SWfMS.

Since the abstract level hides some implementation details, it is simpler and easier to understand than the concrete alternatives presented in Figure 1.b (Workflow#1 to Workflow#3). It is clearer at the abstract level that there is a “Projection” activity using the Cartesian coordinate system approach. We can choose several different programs that can implement it. In this example, at the concrete level, we use, alternatively, the programs: Montage [38] (Workflow #1); Mopex [41] (Workflow #2); and IRAF [42] (Workflow #3) for this same abstract activity. In addition, for the execution, scientists can choose among several different versions of the same program (*e.g.*, Montage1.31 and Montage2.0) to explore possibilities and evaluate different results considering just the abstract Montage activity. Finally, at the execution level, the chosen Montage program in Workflow #1 executions was parameterized with two different values (CRVAL=0.5; CRVAL=0.8), generating two workflow execution instances (Workflow #1.1 and Workflow #1.2). It is worth to mention that we are dealing with only one experiment, represented by one abstract workflow in Figure 1.a. This abstract workflow represents only one possible experiment execution. According to the astronomy experiment, there are two other variations for the activity “Projection”: spherical and celestial coordinate systems. It would be necessary to construct three distinct abstract workflows for the astronomy experiment only to represent all of these variations of the “Projection” activity. Nevertheless, these abstract workflows would not be correlated and we would fall into the same problem of the current approaches that develop several individual concrete workflows for each implementation variation. The same problems can be propagated to the several concrete derivations and another tree of execution derivations as shown in Figure 1.

The common exploration strategy adopted in related work consists in generating several disconnected instances of concrete workflows with different program choices and parameterization for each concrete workflow alternative, also disconnected. This strategy presents several limitations regarding analysis, scalability and maintenance since there is no semantics involved in the workflow and activities representations. For example, it is complex to query mosaic results encompassing variations of Projection activity in the astronomy experiment. Only scientists involved with the experiment conception and

implementation are able to do it, that is, find and manually join all the trials if the scientists can remember the names given to all them. They need to individually query the results of each workflow trial executed in a SWfMS, which is error prone and make their conclusions based on their tacit knowledge. Other scientists that are not directly involved with the experiment may not realize, for instance, that workflows' activities implementing different coordinate systems for image projection (Cartesian, Spherical, and Celestial) have been evaluated in the experiment.

We can group the solution space of MontageExp experiment by important abstract concepts that scientists usually use in a typical provenance analysis. The solution space represents the several implementations of the MontageExp experiment (*i.e.,* concrete workflows) that were created during its lifetime. For example, we can group the implementations by the coordinate systems used in the projection activity or by those that create mosaics without the correction process. However, in traditional solutions, the provenance analysis is done using information directly related to the concrete workflow. It is unfeasible to combine abstract and concrete workflow characteristics in the same provenance query. . For instance, it is unfeasible to analyze the mosaics that were generated in the MontageExp using the Cartesian coordinate system for the projection, since this information is not available when scientists work directly in the concrete level. Additionally, considering a deeper analysis, it is even harder to narrow the analysis to those solutions that created mosaics without correction process. In Section 6, we explore other similar provenance queries.

This astronomy experiment is just a simple scenario – there are many more complex scenarios with several variabilities and restrictions. Even the Montage experiment becomes much more complex in a larger real scale. This is also a reality in other domains such as Computational Fluid Dynamics (CFD) [43]–[45], Uncertainty Quantification in Oil & Gas applications [11] and other scenarios of large-scale experiments (*i.e.,* experiments that manipulate large amounts of data and demand HPC capabilities). Particularly, in the HPC scenario, scientists must analyze many possible implementation alternatives of the experiment just to improve performance. When scientists are not satisfied with a particular workflow composition, they change a specific approach or program for another by modifying the workflow composition. Yet, there are scenarios in which it is necessary to amend parts of the experiment, as in the case of an exchange of a particular technological infrastructure (*e.g.*, moving from clusters to clouds). In such cases, workflows may have to be completely recomposed. In Section 6, we present a case study with an experiment from bioinformatics, which may derive 30 distinct workflows.

4. Related Work

Scientific workflows resemble business processes since they are abstraction models to represent a sequence of steps to reach a specific result. Nevertheless, they are different in several aspects. First, scientific workflows are generally data-centric and are based on dataflow since the activities execution flow is intrinsically defined by the data dependencies among activities. Business processes, on the other hand, are generally based on control flow, *i.e.*, particular control structures (*e.g.*, fork, if, synchronization, start and end point, *etc.*) are used to define the sequence of execution of the activities. Second, business processes are people-centric, usually demanding interaction among different stakeholders interleaved with possibly long idle periods in between these interactions. On the other hand, scientific workflows are usually computationally intensive, requiring fewer human interactions but intensive execution of computational tasks, consuming and producing a much higher amount of data. Finally, business processes are defined according to a thorough vision of the entire process. Once the business workflow is modeled, the process definition is considered final. In science, there are no "final" workflows. Due to the exploratory nature of making science, scientists evaluate the data resulting from the execution of several workflow trials. The satisfactory workflow definition is "final" only for that particular execution with one specific dataset. In this exploratory scenario, the concept of provenance is essential to link the execution resulting data to the workflow abstract concepts. This aspect is not as relevant in the business process management area. They have important tools to represent the process at higher abstraction levels, but they lack an integration of that information to the data generated at execution level, as required in scientific workflows.

Currently, the widely used SWfMS, such as VisTrails [16], Triana [46], Kepler [18], Taverna [19], Pegasus [21], and Swift/T [20], represent workflows only at the concrete level, providing graphical interfaces to help scientists to better visualize their workflows. Each of the aforementioned SWfMS

represents workflows using specific languages and vocabularies that encompass workflow specification, implementation details, environment configurations, *etc*. Workflows expressed in that way are hard to be analyzed even by other scientists from the same experiment domain.

On the other hand, there are some initiatives in the literature [3], [22], [24], [26], [28]–[32] for representing experiments at different levels of abstractions (see Table 1). These abstraction levels are capable of representing different aspects of the experiment, such as provenance, experiment domain concepts and implementation. Some related work [24], [30]–[32] use formal methods to represent these abstraction levels such as ontologies. Others use more pragmatic approaches such as visualization applications [3] or UML diagrams extensions [22]. Archer *et al.* [28] use software product line, similar to ours, to represent the experiment. However, this approach works only at a low level of abstraction, disregarding higher abstract experiment concepts.

GExpLine [47] is a tool that implements the original approach of experiment line conceived by Ogasawara et al. [34]. This tool allows the creation of experiment lines via a graphical interface that can be further derived into concrete workflows. Inspired by a related problem in the software engineering field, the experiment line is an analogy for software product line [48] (SPL for short). SPL are based on two fundamental orthogonal concepts: *variability* and *optionality*. Variability allows for software developers to change and configure components according to a given context. Optionality, it allows a specific component of the SPL to be suppressed when a software system is derived. In SPL, a set of software products that share common characteristics are grouped into a *family*, *i.e.*, they can be developed based on a set of assets. Applying the same concept, experiment lines represent at a high abstraction level a family of scientific workflows that presents common characteristics (*i.e.*, related to a specific scientific domain) in a unique and integrated model. When experiment lines are modeled, they are used to automatically derive abstract and concrete workflows.

GExpLine is the pioneer prototype for modeling experiment lines, but it presents some limitations. The main limitation is the lack of semantics in the experiment line – it cannot make the dataflow explicit in terms of which operations each workflow activity performs and which data is involved. Moreover, the variations expressed in the experiment line model concerns only about concrete level information. We cannot abstract the workflow activity description in terms of approaches or algorithms similar to an abstract workflow. Finally, there is not a provenance approach for tracking the derived concrete workflows to the experiment line. Therefore, we cannot make provenance analysis that aggregates workflows based on specific programs used or data products that are present them.

Lin *et al.* [26] and Plankensteiner *et al.* [29] support different abstraction levels to represent the experiment, but do not present a provenance approach to map these abstraction levels in order to integrate the information and help scientists in the analysis process. Salayandia *et al.* [24] consider this matter only in the higher abstraction levels. However, it is very important to include the lower abstraction levels (*i.e.,* the execution level) such as the works [22], [28], [30]–[32] do. These works enable scientists to run analysis querying data from execution level but referring to abstract data from the experiment conception.

Concepts like variability and optionality used to represent the experiment variation in the workflow definition are not supported by most of the analyzed works. Among the works presented in Table 1, the variability concept is more considered than optionality. Only the work of Salayandia *et al.* [24] explicitly supports the optionality concept to define their workflow definition ontologies (WdOs). WdOS and most of other works based on ontology [24], [30]–[32] represent variability considering experiment abstract concepts. However, some works [3], [28] are limited only to express variability at the concrete level (*e.g.*, parameters for executing a program, configuration of resolution and size of an image, configuration of Web service address ports, etc.). They do not support the use of variability over abstract experiment concepts.

Analyzing the related work we can state that Wings [32] is the closest approach to our work. It can infer variabilities in the experiment definition by using ontologies, but it does not support the concept of optionality. Therefore, scientists cannot explicitly declare, in the abstract workflow specification, which activities can be included or not in the concrete workflow. Wings allows for the definition of conflict relationships, but this is only used to validate the workflow structure. As a result, other relationships such as dependency and conflict cannot be effectively used in the workflow specification to guide the scientist

in the workflow composition. For instance, there is not a mechanism that indicates that if one activity is inserted in the workflow, another activity should be included or removed, as we do in AEL.

Table 1 summarizes the approaches discussed in this section. We can see that even the approaches that share our motivation of defining a model at the experiment level present some limitations (e.g., absence of abstraction level mapping support, focus only in the concrete level, lacking of models to represent the experiment characteristics, etc.). Due to these limitations, scientists have a narrow view of the experiment conception and management. Even when the experiment is documented using higher abstract level models, they are not related to the concrete level. This results in weak provenance analysis and massive use of tacit information that might be lost and unknown for other scientists that want to reproduce the experiment. In Section 5 we discuss how AEL mitigates these issues.

Table 1. Summary of the approaches discussed in related work section

	Experiment Level	Abstraction Levels	Variability	Optionality	Abstract to Concrete Level Mapping
ASKALON [22]	Yes	UML activity diagram; Abstract Grid Workflow Language	No	No	Yes
Archer *et al.* [28]	Yes	Software product line; Concrete workflows	Yes (Concrete level)	No	Yes
GExpLine [47]	Yes	Experiment Line; Concrete Workflow	Yes	Yes	No
IWIR/SHIWA [29]	Yes	Abstract level; Concrete level	No	No	No
Kepler [18]	No	Concrete level	No	No	No
Pegasus [21]	No	Concrete level	No	No	No
Research Object Model [30]	Yes	Ontology; Concrete workflow	No	No	Yes
SADI [31]	Yes	Ontology; Concrete workflow based on web services	Yes	No	Yes
Swift/T [20]	No	Concrete level	No	No	No
Taverna [19]	No	Concrete level	No	No	No
Triana [46]	No	Concrete level	No	No	No
TSL [26]	Yes	Interface layer; Concrete layer	No	No	No
VisMashup [3]	No	Visualization apps; Concrete level	Yes (Concrete level)	No	No
VisTrails [16]	No	Concrete level	No	No	No
WDOs [24]	Yes	Workflow-Driven Ontologies; Semantic Abstract Workflows; Proof Markup Language	Yes	Yes	No
Wings [32]	Yes	Ontology; Concrete workflow	Yes	No	Yes

5. Algebraic Experiment Line

Using the concepts of the software product line approach in the context of scientific experiments, we previously proposed the experiment line approach [34] to model scientific workflows at the experiment level. Experiment line allows to model not only one, but also all possible ways for representing a scientific experiment (variabilities and optionalities intrinsic to a specific experiment), *i.e.,* the workflow derivation space. The experiment line model represents the possible alternatives to model a workflow using an

abstract and uniform notation. This notation can be considered as an abstract workflow since it is not bound to infrastructure issues.

In this paper we reformulate the original proposal of experiment line from Ogasawara *et al.* [34] and present a new approach, called Algebraic Experiment Line. This approach enhances the workflow derivation process by means of using more semantics in the experiment line definition. We incorporate the workflow algebra SciWfA in the experiment line approach. This algebra adds data consumption and production semantics to workflow activities in order to standardize the data model in all abstraction levels. This standardization guarantees the creation of queries that join workflow execution data with their corresponding abstract experiment definition. Additionally, derived workflows can use the algebraic transformations to optimize workflow execution plan. However, since the algebra in [35] is not aware of the experiment line abstractions, we formally define algebraic operations and operands to obtain AEL, an enhanced experiment line approach built on top of the SciWfA algebra. This further improves the potential of runtime optimizations by the workflow engine.

For simplicity, we use the term experiment line instead of algebraic experiment line to present the main concepts of the approach along the text. Therefore, from now on, anytime the term experiment line is mentioned in the text please refer to the AEL approach. The remaining of this Section is organized as follows. Section 5.1 presents an overview of the proposed approach. Section 5.2 formalizes the experiment line approach. Finally, Section 5.3 presents a specification of a tool for supporting the creation of experiment lines.

5.1. Overview

An experiment line implements the experiment abstraction level. The relation of an experiment line to the traditional abstract and concrete workflow abstraction levels defined in the literature can be described as follows. The experiment line (*i.e.,* the higher abstract level) represents all *n* possible ways to conduct and represent a scientific experiment, according to the hypothesis of the scientist. Based on experiment lines, workflows can be derived for the experiment through the concepts of variability, optionality, derivation, and instantiation. Variability means that for a specific activity in an experiment line there is more than one abstract alternative (*e.g.*, there is more than one approach that could be applied to perform the task denoted by that activity). The concept of optionality, on the other hand, defines that a specific activity of the experiment line can be suppressed when a workflow is derived. In an experiment line, each activity is associated to an operator in the workflow algebra (see Section 2), and to input and output schemas. Although experiment lines are agnostic of implementation, associating experiment line activities with algebraic operators exposes the semantics (represented by such operators) behind each activity at this abstraction level.

The experiment line model is similar to a workflow model, which defines the main flow of activities, with an inclusion of a set of annotations that define the multiple variations of the workflow activities. Using the experiment line model, scientists are able to derive workflows that represent a specific cutting of an experiment line. In this cutting, scientists choose specific approaches from (possibly) several that are specified to the activities. These derived workflows represent the second abstraction level, where the workflow activities inherits their corresponding algebraic operators defined in the experiment line, leading to the workflow representation by means of algebraic expressions. We call this derived workflow an *abstract workflow*.

Finally, the concrete workflow (*i.e.,* the concrete level) takes these algebraic expressions and implements them in a specific SWfMS with a specific workflow language, *i.e.,* it instantiates the abstract workflow activities to computational resources. This process is called instantiation. Summarizing, we have two main processes for transforming an experiment line into a final concrete workflow that is going to be executed in a specific SWfMS: **derivation** and **instantiation**. The derivation process takes as input an experiment line, and produces an abstract workflow. In other words, in the derivation process, there is a cutting of the execution approach space of the experiment line workflow, generating a valid and unique execution approach, *i.e.*, an abstract workflow (containing algebraic expressions). The instantiation process takes as input an abstract workflow and produces a concrete one. More specifically, abstract workflow activities are instantiated in terms of computational resources, generating a concrete workflow.

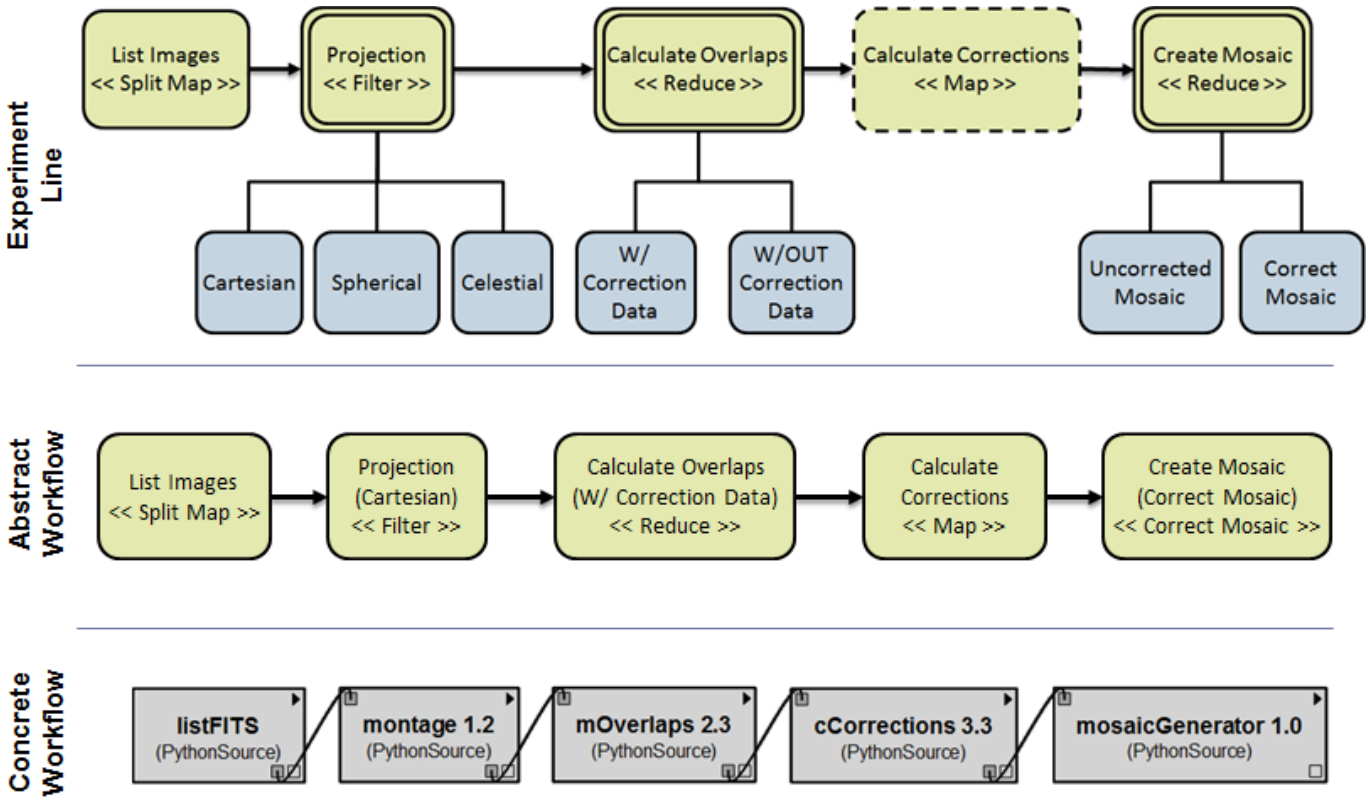


Figure 2. An example of experiment line in the context of an astronomy data analysis experiment.

Figure 2 presents an example of an experiment line in the context of an astronomy data analysis experiment, presented in Section 3, with abstract and concrete workflows derived from it. The notation used to represent the experiment line is the one we implemented in the ExpLine tool (for details, see Section 5.3). In the experiment line, the activities List Images, Projection, Calculate Overlaps, and Create Mosaic are mandatory (*i.e.*, they must be present in any derived abstract workflow). Mandatory activities are represented by solid line rectangles. On the other hand, the activity Calculate Corrections is optional and can be present or suppressed in the derived workflow, depending of what kind of algorithm was selected in the workflow activity Calculate Overlaps, as described in Section 3. Optional activities are represented by dashed line rectangles. The experiment line variabilities are represented by variant activities and they are grouped by variation point activities. For instance, Cartesian, Spherical, and Celestial are variants of the variation point activity Projection. Variation points are represented by double-line rectangles, while the variants are represented by blue rectangles. Activities that have no variants are called invariants (*e.g.*, List Images, Calculate Corrections.).

It is worth mentioning that each abstraction level uses the workflow algebra SciWfA as the foundation to represent the experiment workflow. As described in Section 2, each workflow activity is associated to an algebraic operator according to how it consumes and produces data (represented by relations). Therefore, when a workflow activity in the experiment line model is derived into an abstract workflow, and then instantiated to a concrete workflow, it inherits the algebraic information of SciWfA defined in the experiment line (*i.e.*, the algebraic operator and the input and output relation schemas). This is important to guarantee the compatibility between the abstraction levels and to link provenance. For example, in Figure 2, the activity List Images from the abstract workflow inherited the algebraic operator “Split Map”, which was defined in the corresponding workflow activity in the experiment line. The abstract workflow activity List Image is implemented by the program listFITS in the concrete workflow. As described in Section 3 (Figure 1), the program listFITS behaves according to the Split Map operation.

5.2. Experiment Line Formalism

With the purpose of clarifying and validating the experiment line approach, this section presents a formalization of the approach. This formalization is organized in five sections described as follows. Section 5.2.1 defines the basic concepts related to the experiment line. Section 5.2.2 introduces the

functions used in the workflow derivation process. Section 5.2.3 describes the workflow activity types present in the experiment line. Section 5.2.4 presents the composition rules. Finally, Section 5.2.5 discusses how to validate workflows derived from an experiment line.

5.2.1. Basic Concepts

Definition 8. (EXPERIMENT LINE) An experiment line *el* is represented by a triple $<AEL, DepEL, Rl>$, where AEL corresponds to the set of activities in an experiment line, denoted by $AEL = \{ael_1, ..., ael_n\}$; *DepEL* represents the set of edges of the experiment line, denoted by $DepEL = \{depel_1, ..., depel_m\}$, and *Rl* is the set of rules that constrains the possible workflow derivation, denoted by $Rl = \{rl_1, ..., rl_o\}$.

It is worth mentioning that each ael_i inherits the workflow activity definition from Definition 1 (Section 2.1) and is associated to one of the algebraic operators defined in Section 2.2.

Optionality and Variability are the basic concepts to represent variation in the experiment line. Optionality is a concept that defines if an experiment line activity can be suppressed or not when a workflow is derived. Variability is a concept that defines if an experiment line activity can be represented by more than one abstract alternative (*e.g.*, more than one approach that could be applied to perform the task denoted by that activity). Their definitions are described as follows.

Definition 9. (OPTIONALITY) An experiment line activity *ael* can be classified as mandatory, when its presence is required in all derived workflows, or optional, when the workflow can be derived without its presence. The set of all mandatory activities is denoted by $MA \mid MA \subseteq AEL$, and the set of all optional activities is denoted by $OA \mid OA \subseteq AEL$. Thus, $AEL = MA \cup OA$ and $MA \cap OA = \emptyset$.

Definition 10. (VARIABILITY) An experiment line activity *ael* can be classified as a variation point iff there is more than one abstract activity associated with it. Otherwise *ael* is classified as invariant (when it can be represented by a singular abstract activity). The set of all variation point activities of an experiment line *el* is denoted by $VPA \mid VPA \subset AEL$, and the set of all invariant activities is denoted by $IA \mid IA \subseteq AEL$. Additionally, each activity $vpa_i \mid vpa_i \in VPA$ has a set of approach variations (variants) $VA \mid VA \subset vpa_i$. Thus, $AEL = VPA \cup IA$ and $VPA \cap IA = \emptyset$.

Definition 11. (ABSTRACT WORKFLOW) An abstract workflow *aw* is a valid workflow (see Definition 7 from Section 2.1) represented by the tuple $<AA, ADep>$, where *AA* is the set of abstract workflow activities, denoted by $AA = \{aa_1, ..., aa_n\}$; and *ADep* corresponds to the set of edges of the abstract workflow, denoted by $ADep = \{adep_1, ..., adep_m\}$.

It is worth mentioning that $AA \subseteq AEL$, where *AEL* is the set of activities of an experiment line *el*. Each abstract workflow aw_i derived from an experiment line *el* is unique and their activities (*i.e., AA*) are associated to specific operators in the workflow algebra. The function $Oper(aa_j)$ returns the operator associated to a specific abstract activity (*i.e.*, its semantics). A derivation of an experiment line *el* is performed by cutting its derivation space, forming a derived abstract workflow. This derivation is based on a derivation function (presented in Definition 12).

5.2.2. Derivation and Instantiation Functions

Definition 12. (DERIVATION FUNCTION) The derivation of an abstract workflow *aw* from an experiment line *el* is represented by a function called *DerivationFunction(el, dp)*, such that $DerivationFunction(el, dp): el \rightarrow aw$, where *el* is the experiment line, *aw* is the target abstract workflow, and *dp* is the derivation plan that describes which elements are selected in the experiment line.

Definition 13. (INSTANTIATION FUNCTION) The instantiation of a concrete workflow *cw* from an abstract workflow *aw* is represented by a function called *InstantiationFunction(aw, swfms)*, such that $InstantiationFunction(aw, swfms): aw \rightarrow cw$, where *aw* is the abstract workflow, *cw* is the target concrete workflow, and *swfms* is the target scientific workflow management system where the concrete workflow will be instantiated.

Definition 14. (VALID WORKFLOW DERIVATIONS) All valid derivations of an experiment line *el* (*i.e.*, the derived abstract workflows that respect the rules described in Definition 22 – presented next) are represented by the set $AW = \{aw_1, ..., aw_n\}$.

For simplicity, from now on when workflow activities are mentioned in this article with no abstraction level modifier please refer to activities of an experiment line. At the abstract and concrete levels, activities are mentioned with their respective adjectives, *i.e.,* abstract activities and concrete activities. Figure 2 illustrates an abstract workflow derived from the experiment line. In the abstract workflow, the Cartesian approach was selected for the variation point activity Projection and the optional activity Calculate Corrections was included.

Once the relationships between experiment line and abstract workflow were explained by introducing the concepts of variability and optionality, we now discuss with more detail the different activity types originated from these two concepts.

5.2.3. Workflow Activity Types

Definition 15. (MANDATORY ACTIVITY) Given the set $AW = \{aw_1, ..., aw_n\}$, which represents all the valid abstract workflow derivations from an experiment line *el*, and the set $MA = \{ma_1, ..., ma_m\}$, which represents all mandatory activities of an experiment line *el*, we have $\forall aw_i \in AW, MA \subset aw_i.AA$.

Definition 16. (OPTIONAL ACTIVITY) An optional workflow activity may or may not be present in any derived abstract and concrete workflows. A minimal condition for an activity to be optional is to have the schema preservation characteristic, i.e., the workflow activity input and output schemas (see Section 2.1) must be the same.

Definition 17. (INVARIANT ACTIVITY) An invariant activity *ia* is an activity that presents only one implementation solution.

Invariant activity can be considered a particular case of a variation point that has only one variant activity. Hence, when an abstract workflow is derived it can be automatically selected if it is also classified as a mandatory activity.

Definition 18. (VARIATION POINT ACTIVITY) A variation point activity *vpa* is an activity that is associated to a set of variant activities $VA \mid VA \subset vpa$. A *vpa* should be represented by one and only one variant $va_i \mid va_i \in VA$ in a derived abstract workflow *aw*. In other words, considering *AA* as the set of activities of the abstract workflow *aw*, we have $\exists! \, va_i \in VA \mid va_i \subset AA$.

It is worth mentioning that *vpa* is a unique and exclusive element of the experiment line *el*. The abstract activities AA of a derived abstract workflow *aw* contain only the variant activity *va* selected for a specific variation point activity *vpa*. Therefore, the abstract workflows *AW* derived from AEL must comply with the following constraint: $\forall aw_i \in AW, \ \forall vpa_j \in VPA \mid vpa_j \not\subset aw_i$.

Definition 19. (VARIANT ACTIVITY) A variant activity in an experiment line *el* represents one of the different approaches of a variation point activity. A minimal set of conditions for a workflow activity to be a variant of a variation point activity is:

- The input relation set of the variation point must have the same schema of the input relation set of the variant activity;
- The output relation of the variation point must have the same schema of the output relation of the variant activity.

It is worth mentioning that the dependencies *dep* of the variation point activity to the other activities defined in the experiment line *el* ($dep \subseteq el.DepEL$) is the same, no matter which variant activity *va* is selected, since the input and output schemas are always preserved.

Additionally, optionality and variability are orthogonal concepts. Therefore, experiment line activities are classified based on these two concepts. For example, if an experiment line activity is categorized as

invariant (*i.e.*, variability characteristic) it can also be categorized as optional or mandatory using the variability classification. In the case of variant activities, they are not classified based on optionality characteristics. This is due to the fact that they do not belong to the experiment line workflow definition but exclusively to the variation point definition.

5.2.4. Composition Rules

With the help of variability and optionality concepts (represented by the different activity types), it is possible to model a large set of experiment lines. However, some characteristics such as conflict and dependency relationships between activities in the experiment line cannot be expressed using only these activity types. Therefore, in the experiment line we have rules that allow scientists to model more sophisticated and customized relationships.

Definition 20. (RULE) An experiment line *el* has a set of composition rules $Rl = \{rl_1, ..., rl_n\}$. A rule rl_i is composed of a logical implication of the type $\alpha \rightarrow \beta$, where α is the premise and represents absence and presence conditions of activities in the derived abstract workflow *aw*, and the consequent β (or conclusion) represents insertions or removals of workflow activities in *aw* (*e.g.*, $oa_i \wedge va_j \rightarrow oa_k$).

A rule rl_i is valid when it restricts the experiment line selection (cutting) to a valid workflow composition, *i.e.*, it does not violate any other rule rl_j or any experiment line basic definition (*e.g.*, invariant, variant, variation point, mandatory, and optional rules). Additionally, the workflow activities used in an implication must be optional or variants in an experiment line *el* (*e.g.*, a rule rl_i is not allowed to remove a mandatory workflow activity in the abstract workflow).

Definition 21. (VALID RULE) A rule *rl* is considered valid iff $ValidRule(el, rl) \leftrightarrow (Optional(el, a_i) \vee Var(el, a_i)), \forall a_i \mid a_i \in RuleActivities(rl)$. The function *RuleActivities(rl)* returns the workflow activities used in the rule.

In the astronomy experiment line of Figure 2, in order to create corrected mosaics (i.e. select the variant activity Corrected Mosaic for the variation point activity Create Mosaic) it is necessary to calculate the image corrections (*i.e.*, include the optional activity Calculate Corrections). The inclusion of Calculate Corrections also implies that during the image overlap calculus the correction data should be processed (i.e., select the variant activity W/ Correction Data for the variation point activity Calculate Overlaps). Therefore, we have two rules: Rl_1 (Corrected Mosaic → Calculate Corrections) and Rl_2 (Calculate Corrections → W/ Correction Data). The derived abstract workflow should meet these rules to be valid.

5.2.5. Validation of Derived Workflows

The SciWfA workflow algebra is used to verify how a specific activity can consume and produce data. As described in Section 2.1, the input and output relations of the workflow activities represented in the SciWfA workflow algebra follow their respective schemas. This way, by using the workflow algebra we can support derivation by generating concrete workflows whose programs are compatible.

Besides verifying compatibility among programs, we have to validate derived workflows. The process of validating a derived workflow has to take into account all composition rules defined by scientists (or workflow modelers). For example, it must verify the inclusion of mandatory activities, the choice of a single variant for variation point activities, and so forth. Moreover, the additional composition rules (*i.e.*, the set *Rl*) associated to the experiment line should be analyzed to check if they are met in the derived abstract workflow.

Definition 22. (VALIDATION OF WORKFLOW DERIVATION) An abstract workflow *aw* derived from an experiment line *el* is valid iff:

- *aw* is a valid workflow, according to Definition 7 (Section 2.1);
- all the mandatory activities defined in *el* is present in *aw*, according to Definition 15;
- for each selected variation point activity *vpa*, a unique variant activity *va* is selected, according to Definition 18;

- *aw* respects all the dependencies defined in *el*;
- *aw* respects all the rules defined in *el*.

5.3. ExpLine: Architecture and Provenance Model

In this Section we present a tool for supporting the creation of experiment lines, which derives workflows in different levels of abstraction. This tool, named ExpLine, is complementary to existing SWfMS, thus offering extra higher abstract representation layers to be coupled to the existing approaches. Its goal is to provide an environment for scientists to model their scientific experiments using an abstract level language to automatically generate corresponding executable workflows in pre-defined SWfMS such as VisTrails, Kepler, or SciCumulus.

Figure 3 presents the architecture of Expline illustrating its main execution process and associated components. The core architecture of ExpLine is composed of three components: Interface Module, Derivation Module, and Instantiation Module. From these three components, the experiment line is modeled (Interface Module), abstract workflows are derived from the experiment line (Derivation Module), and concrete workflows are instantiated in specific SWfMS (Instantiation Module).

The Derivation module uses a knowledge base to store the experiment line general rules and concepts to be further processed during the workflow derivation. The knowledge base is implemented in Prolog [49] and so the Definitions presented in Section 5.2 were translated into Prolog facts and rules. Additionally, scientists can create composition rules (Definition 20 and Definition 21) in the interface module, which are also translated into Prolog and stored in the knowledge base. When the knowledge base is completely populated, it is used by the Derivation Module to calculate the implications on selecting or unselecting one activity during the workflow derivation process.

The Instantiation module is based on the concept of cartridges [50], *i.e.,* when scientists want to instantiate the abstract workflow generated from the experiment line into a concrete workflow, they have to choose one cartridge in a set of available cartridges. Each cartridge generates concrete workflows for a specific representation language (*e.g.,* XML, SCUFL, SwiftScript) of a specific workflow engine (*e.g.*, SciCumulus, VisTrails, and Taverna).

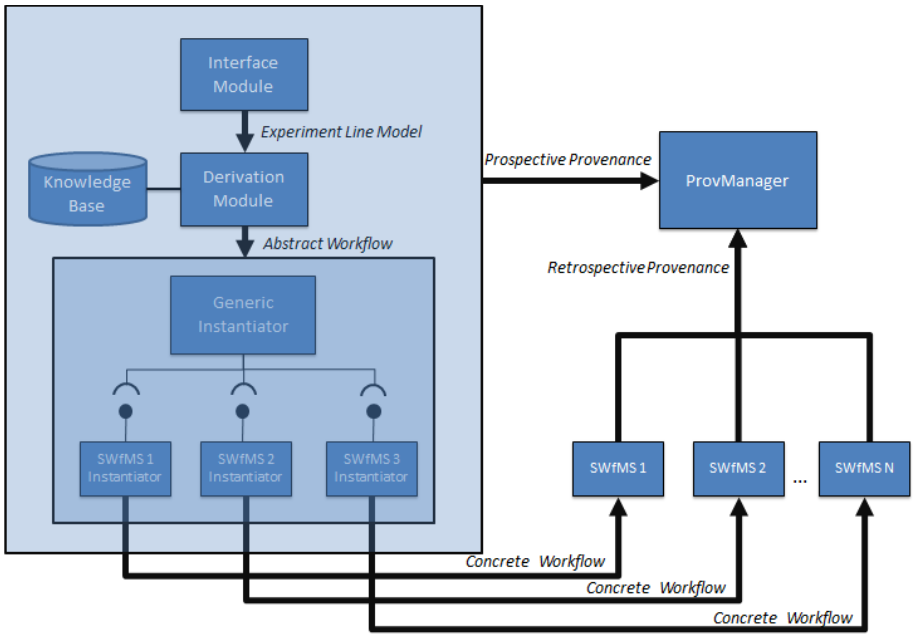


Figure 3. ExpLine architecture

Another important component of ExpLine architecture is related to provenance management, which is dealt by the system named ProvManager [51]. ProvManager manages, in an integrated way, provenance data that are distributed in several workflow systems but are part of the same experiment context. ProvManager was originated to deal with experiments that during their life cycles are executed by several workflows systems or, in a worst scenario, use a combined solution of different workflow systems where

each one executes a segment of the experiment. ProvManager abstracts the technological complexity of different and heterogeneous workflow systems and integrates the experiment provenance data enabling scientists to visualize them in a unique and standard interface. ProvManager suits to the characteristics of ExpLine since it is not possible to know in advance which workflow system will be selected by the scientist to execute the derived workflows. Additionally, the scientist can interchange to different workflow systems during the experiment life cycle. ProvManager enables scientists to freely do that without concerning about losing provenance data.

In ExpLine, ProvManager collects and manages prospective and retrospective provenance data from the experiment line using abstract/concrete information. Prospective provenance in ExpLine consists of information about the experiment line, abstract and concrete workflows, and workflow components such as activities, relations, and schemas. Retrospective provenance consists of information associated to the execution of the experiment (*i.e.*, derived workflow execution). ProvManager acts in three important moments. In the composition, when the experiment line is modeled, ProvManager collects prospective provenance information. The second moment is in the workflow derivation and instantiation. At this moment, ProvManager configures the derived workflows via instrumentation [51] to collect retrospective provenance information outside of ExpLine when they will be executed in the workflow system. The third moment is the publication of retrospective provenance information to ProvManager from the instrumented workflow activities that are executed in the workflow system.

Figure 4 presents an ExpLine provenance model, which is used in ProvManager to support the experiment line concepts. ExpLine provenance model is designed to represent provenance data from experiment lines that can derive concrete workflows for different SWfMS. All data can be queried at runtime [52]. ExpLine model is an extension of PROV-Wf that was recently proposed [52], which is based on the PROV W3C recommendation [53], to represent the experiment line abstractions. PROV allows for representing entities, people, and processes involved in the generation of a piece of data so that further extensions can be defined.

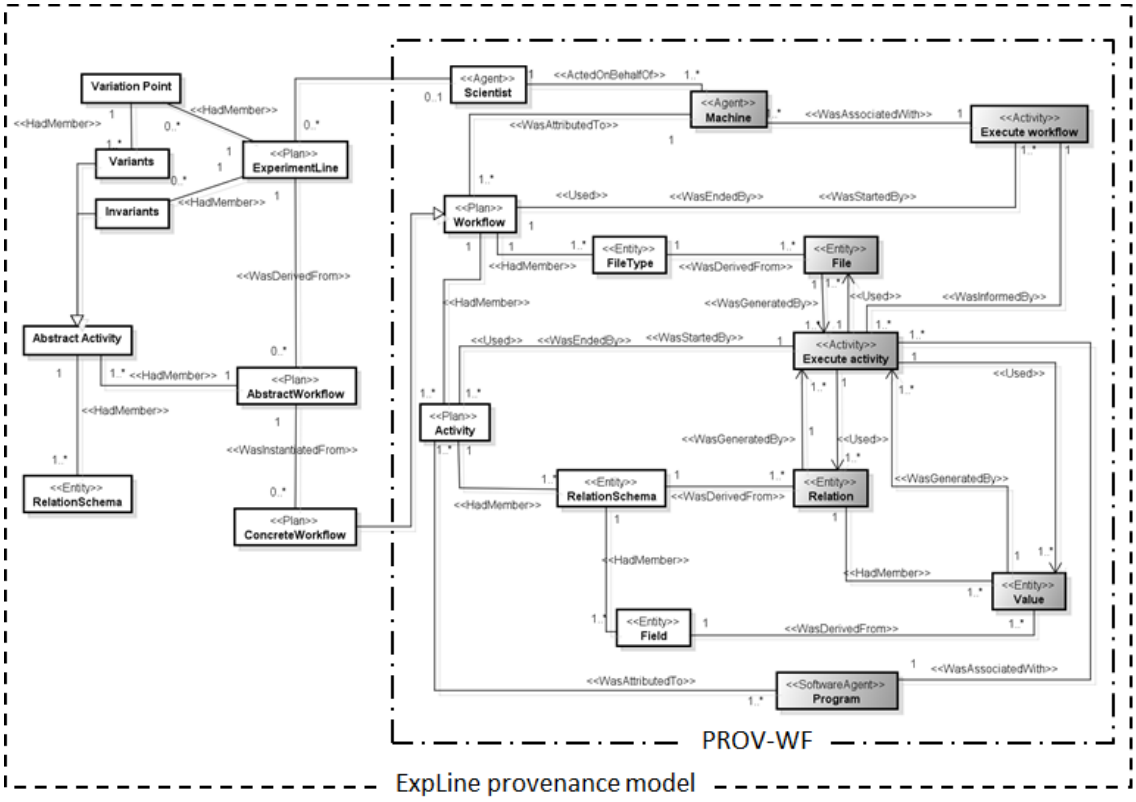


Figure 4. ExpLine provenance model

The ExpLine provenance model is composed of two main parts: the structure of the experiment (white classes in the UML class diagram), and the execution of the experiment (gray classes). Furthermore, the stereotypes in the UML class diagram are used to represent PROV components. The agent *Scientist (i.e., <<Agent>> Scientist)* represents a person that models and executes the experiment line. The entity

ExperimentLine (*i.e.,* <<Plan>> *ExperimentLine*) is composed of a set of activities, where each activity is responsible for representing variabilities and optionalities in the experiment line level. Experiment lines can be derived (*i.e.,* <<Activity>> *Derivation*) into workflows. The entity workflow (*i.e.,* <<Plan>> *Workflow*) is composed of a set of activities (*i.e.,* <<Plan>> *Activity*). Each activity in a workflow is responsible for representing an approach (abstract level) or a program (concrete level). To express all data that is consumed and produced by execution instances (and to verify compatibilities and to add semantics), the entity *RelationSchema* is associated with a schema and can be defined with multiple fields. Each field (*i.e.,* << Entity>> *Field*) describes the meaning of each parameter associated to a program that is associated to an *Activity*. The other elements (gray classes) are associated to the execution of the experiment. Although they are very important information, detailing them transcends the scope of this work.

6. Case Study

The AEL approach proposed in this work is supported by a formalism that is reflected in a combination of tools. Besides modeling an experiment line, scientists have to choose a specific SWfMS to derive their concrete workflows. The goal of these tools is to aid scientists at each step of the process — from the experiment line composition to the configuration of each of its concrete executable workflows. This section presents a case study in the phylogenomic area that can be executed using a prototype of ExpLine to model the experiment lines and the SciCumulus system [17] as the SWfMS.

SciCumulus is a workflow engine that distributes and monitors the parallel execution of workflow activities dispatched from a local machine into a cloud environment, such as Amazon EC2 [54]. Differently from other SWfMS and parallel implementations such as Hadoop [55], a *de facto* standard MapReduce [37] implementation, SciCumulus is able to execute in two different modes: static and adaptive. In the static mode, SciCumulus generates a scheduling plan based on a static scenario of the environment at the beginning of the workflow execution, *i.e.,* it only considers the amount of virtual machines and the state of all virtual machines in the beginning of a workflow execution. In the adaptive mode, on the other hand, SciCumulus analyzes the current status of the environment (*i.e.,* the real capacity of each virtual machine), by querying provenance data, and the performance restrictions (imposed by scientists). Then it dynamically scales the number of VMs used in the execution up and down according to the changes and availability of resources in the cloud environment, aiming at maintaining the overall performance of the workflow. For more details about SciCumulus, please refer to Oliveira *et al.* [56].

6.1. Phylogenomic Analysis Experiment

Phylogenomics is a bioinformatics domain that uses large genomic data sets to address difficult phylogenomic[1] problems. Alternatively, phylogenomics may be described as the use of phylogeny and comparative analysis to infer biological events of genome evolution [57]. Phylogenomic experiments demand HPC capabilities since they are data and computing-intensive. Phylogenomic experiments produce phylogenomic trees that depict existing evolutionary relationships and are used for inferring phylogeny. The Phylogenomic experiment presented by Oliveira *et al.* [58] has one example of a workflow that processes a large collection of multi-fasta files (*e.g.,* DNA, RNA or protein sequences) to obtain phylogenomic trees.

Figure 5 presents the experiment line for the phylogenomic analysis experiment. The workflow (represented by the yellow elements) is composed of seven activities. The first four activities are specific for the phylogenetic analysis: MSA (Multiple Sequence Alignment) construction, evolutionary model election, phylogenetic tree construction, and MSA conversion. The last three activities represent the phylogenomic analysis itself: MSA concatenation, evolutionary model selection, and phylogenomic tree construction. In the end of the workflow, there is still a manual activity for electing the best evolutionary tree, which is done by the scientist. All of these activities, excluding the manual one, are either computing-intensive or data-intensive. For more details please refer to Oliveira *et al.* [59].

[1] Phylogenomics is the study of evolutionary relationships among groups of organisms – complete genes (*e.g.*, species).

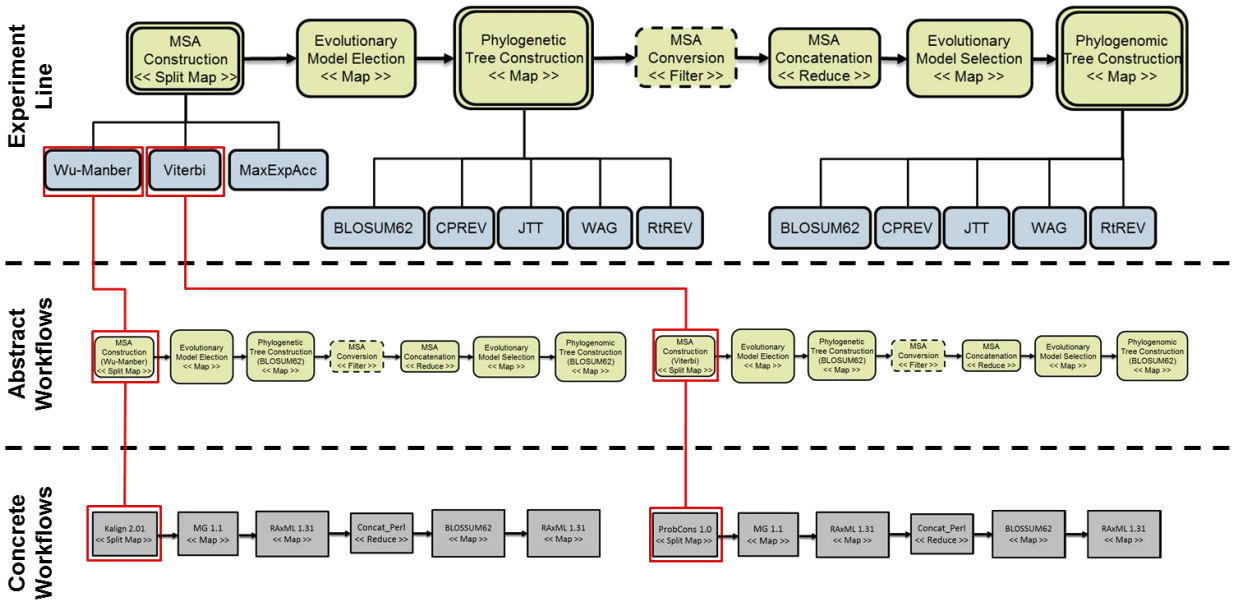


Figure 5. Phylogenomic experiment line

The activity "MSA Construction" uses a specific MSA program to produce individual multiple sequence alignments. There are several available programs that implement this activity (*e.g.,* ClustalW, Kalign, MAFFT, Muscle or ProbCons). Each one of these programs follows a specific bioinformatics algorithm (*i.e.,* approach) (*e.g.,* maximum expected accuracy, Viterbi alignment, or Wu-Manber string-matching [60]), which clearly provides different performance and results that directly impacts on the choice of them. In addition, scientists do not know *a priori* which MSA algorithm provides the best results for a specific input, so they have to try different programs in different workflow executions (trials). Each MSA program receives a multi-fasta file as input and produces a MSA as output.

Each MSA is then tested by activity "Evolutionary Model Election" to find out the best evolutionary model (BLOSUM62, CPREV, JTT, WAG, and RtREV) using ModelGenerator. Activity "Phylogenetic Tree Construction" generates phylogenetic trees using a maximum likelihood implementation that varies its execution according to the selected evolutionary model. Activity "MSA Conversion" converts each MSA to the PHYLIP format [61]. Although useful, this conversion is optional. This way, scientists have to decide if the PHYLIP conversion is really needed. Next, all individual MSA (converted or not) are concatenated to obtain a "super alignment" in activity "MSA Concatenation". In activity "Evolutionary Model Selection", a specific evolutionary model is selected and is used in activity "Phylogenomic tree construction" to construct phylogenomic trees consuming each "super alignment". Finally, scientists have to choose one evolutionary model between five possibilities. All phylogenomic trees have to be analyzed to elect the one that best reflects the organisms' evolutionary relationships hypothesis. If we consider that scientists have three different MSA algorithms, five different evolutionary models, and the optionality in activity "MSA format conversion", there are 30 different ways of obtaining phylogenomic trees. In other words, we have 30 different workflows that can be modeled for a single phylogenomic analysis experiment (*i.e.,* 3 MSA × 5 evolutionary model × 2 options for MSA conversion activity).

6.2. Modeling and Deriving a Phylogenomic Analysis Experiment Line

In phylogenomic experiments it is common to explore several available alternatives of algorithms to investigate which one produces the best results. This way, scientists have to explore all alignment algorithms implemented in different forms. In addition, in several cases, scientists explore different programs for the same algorithm to evaluate which one presents better data quality × performance ratio as presented by Ocaña *et al*. [8].

Therefore, based on the activities presented in **Figure 5**, the first step is to elicitate all variabilities and optionalities that are part of the experiment line. For example, in the phylogenomic experiment line, the "MSA Construction" activity is a variation point, which is associated to three variant activities Wu-

Manber, Viterbi, and MaxExpAcc. Each one of these activities represents a specific approach that scientists have to choose to derive their workflows. In fact, scientists do explore each one of these variations. When using directly a SWfMS instead of first using ExpLine, all generated concrete workflows are managed independently, *i.e.,* they are not related.

The second step to be performed is to model the experiment line. It can be created in a top-down way from the highest abstract level to the concrete level. Scientists define the variabilities, optionalities and the composition rules. In special, composition rules are very important in this phylogenomic experiment to restrict and dictate how the variabilities and optionalities are selected. Some MSA approaches used in this experiment are incompatible with particular evolutionary models. The experiment line is modeled just once and scientists derive workflows to be executed or make annotations any time after that. From the modeled experiment line, scientists may conduct experiments by deriving and instantiating, respectively, abstract and concrete workflows associated to that experiment. In this case study we explore two different compositions for the experiment. In the first one, scientists choose the Wu-Manber approach as variant activity to the "MSA Construction" variation point. In the second one, scientists choose the maximum expected accuracy approach (MaxExpAcc) as variant activity. These choices lead to two different valid abstract workflows. In this case, MSA approaches, Wu-Manber and MaxExpAcc, are compatible with the selected evolutionary model Blossum62. The same does not happen if the evolutionary model JTT were selected. Wu-Manber is not compatible with that model. **Figure 5** illustrates a preview of these two abstract workflows instantiated in SciCumulus (*i.e.,* two concrete workflows), which were derived from the phylogenomic analysis experiment line (modeled in ExpLine).

In **Figure 5**, we only present the first activities of the concrete workflows for the sake of simplicity. One of them uses Kalign for the first activity and the other uses ProbCons. These programs are associated with Wu-Manber and MaxExpAcc algorithms, respectively. These two variations of the same experiment produce results with different quality and execution performance. For example, if we take the phylogenomic experiment conducted by Oliveira *et al*. [56], we can observe that the concrete workflow variation that uses the Kalign program is more efficient in execution time than the ProbCons variation, as is shown in Figure 6.a.

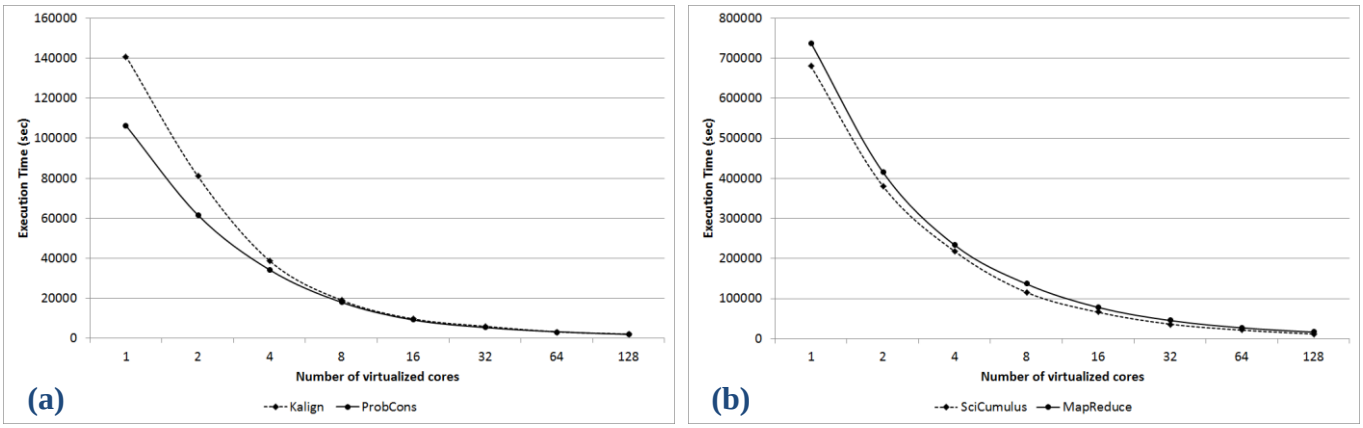


Figure 6. Execution time using (a) Kalign and Probcons and (b) using our derived algebraic workflow in SciCumulus and a Hadoop implementation as presented by Oliveira *et al* [56]

To analyze the quality of the results we compared the signalized set of phylogenomic trees using tree distance, reinforcing phylogenetic inferences. The idea is to analyze which results presents bootstrap replication values greater than the other trees. Using this test, we concluded that these two methods provide trees with different quality. In this experiment, Kalign produced phylogenomic trees with more quality than ProbCons. Since the scientific workflow that contains Kalign produced results with more quality and executed faster than the workflow that contains ProbCons program, it is natural that scientists may opt to use Kalign in future larger executions. In this case, the experiment line helped scientists to explore a space of options to conduct their experiments.

Once the phylogenomic analysis experiment was modeled using experiment line, the experiment line activities and derived abstract workflows are directly associated to an algebraic operator and the dependencies are represented in the algebraic expression. Therefore, the abstract workflows derived from

the phylogenomic analysis experiment line are execution-optimization-ready. This capability is only useful if the abstract workflow is instantiated and executed in a SWfMS that supports the SciWfA workflow algebra (*i.e.,* SciCumulus).

In the phylogenomic analysis experiment line, the activity "MSA Conversion" is associated with the filter operator. A filter operator in our workflow algebra works similar to the selection operator (σ) in the relational algebra [62]. It has the purpose of reducing the amount of data to be processed. This way, if the optimizer is able to anticipate the execution of "MSA Conversion", the execution may benefit from data reduction in an earlier stage of the scientific workflow. In the concrete representation, this results in an anticipation of the execution of "MSA Conversion" (executed by ReadSeq program). This means it is executed before the activity "Evolutionary Model Election" (executed by ModelGenerator program version 1.1) and not right before the activity "MSA Concatenation" (executed by an in-house PerlScript) as initially defined. In previous evaluation [56], the execution of the phylogenomic experiment implemented in our approach (*i.e.*, SciCumulus optimizing and distributing the execution in a cloud environment) was compared to a traditional Hadoop implementation. The gain was about 10% of performance when using our workflow algebra. Figure 6.b shows the corresponding execution performance results of the two approaches. The workflow execution optimization issue was explored with more detail in the evaluation presented in Oliveira *et al.* [56].

Additionally, scientists can perform queries in the provenance model of ExpLine to analyze the results of the entire experiment. Querying provenance corresponding to all workflow executions within the same experiment line would require a significant effort when using only the SWfMS provenance support. Since the main abstraction in the existing SWfMS is the workflow itself, it is not simple to gather and combine provenance results from distinct workflow executions. On the other hand, when using ExpLine, these queries are directly formulated. For example, scientists may want to discover which MSA approach is more used in the derived abstract workflows. Figure 7 illustrates the SQL statement for this query, which returns Viterbi, Wu-Manber, and MaxExpAcc (in this order) as the most used MSA approaches.

```
SELECT abstActv.Name,  count(abstActv.Name)  as count
FROM AbstractActivity abstActv, AbstractWorkflow abstWf, ExpLine el, Derivation der, VariationPoint vp, Variant var
WHERE el.Name  = 'Phylogenomic' AND vp.Name  = 'MSA Construction'
      AND der.ExpLineID  = el.ID AND der.AbstractWorkflowID = abstWf.ID AND abstActv.ID  = der.AbstractActivityID
      AND vp.ExpLineID  = el.ID AND vp.ID = var.VariationPointID AND var.AbstractActivityID  = abstActv.ID
GROUP  BY abstActv.Name;
```

Figure 7. Example of SQL statement and respective result table for Phylogenomic experiment

As an additional example, scientists may want to analyze which MSA approach led to the best execution meantime. This can be queried by gathering the associated execution time of all concrete workflow execution (*Table ExecutionWorkflow*) from the phylogenomic experiment (*Table Experiment*). Then, the results are grouped by each variant activity (*Table Variant*) from the variation point (*Table VariationPoint*) "MSA Construction". Figure 8 illustrates the SQL statement for this query processed in the phylogenomic experiment database. Wu-Manber was the fastest approach with the execution meantime of 8.1 hours, while Viterbi and MaxExpAcc presented, respectively, the results of 8.5 and 9.1 hours.

```
SELECT abstActv.Name,  avg(eWf.endTime-eWf.startTime)  as meantime
FROM AbstractActivity abstActv, AbstractWorkflow abstWf, ExpLine el, Derivation der, VariationPoint vp, Variant var,
ExecutionWorkflow eWf, ExecutionActivity eActv, ConcreteWorkflow concWf, ConcreteActivity concActv, Instantiation inst
WHERE el.Name  = 'Phylogenomic' AND vp.Name  = 'MSA Construction'
      AND vp.ExpLineID  = el.ID AND vp.ID = var.VariationPointID AND var.AbstractActivityID  = abstActv.ID
      AND der.ExpLineID  = el.ID AND der.AbstractWorkflowID = abstWf.ID AND der.AbstractActivityID = abstActv.ID
      AND inst.AbstractWorkflowID  = abstWf.ID AND inst.ConcreteWorkflowID = concWf.ID
      AND inst.AbstractActivityID  = abstActv.ID AND inst.ConcreteActivityID = concActv.ID
      AND eWf.ConcreteWorkflowID = concWf.ID AND eActv.ConcreteActivityID  = concActv.ID
      AND eActv.ExecutionWorkflowID  = eWf.ID
GROUP  BY abstActv.Name;
```

Figure 8. Workflow execution meantime for each MSA approach

The provenance model of ExpLine can also be used to guide scientists in the workflow composition. Scientists can use the execution data collected of all the workflows derived from the experiment line to analyze which derived workflow better fits to a particular scenario. For instance, considering an execution scenario, a scientist may want to derive a workflow from Phylogenomic experiment to be executed in a cloud or a grid environment. The main interest in this trial is to compose a workflow that results in a minimal cost when executed in the environment. A minimal cost could consider the number of execution nodes and the total amount of execution time. Based on the execution history, scientists may conclude that workflows that use Wu-Manber approach for the MSA activity and BLOSUM62 as the evolutionary model are statically better than other combinations. This selection process could be incorporated in the workflow engine optimizer to become automatic during the workflow execution. The workflow engine optimizer could adapt the workflow execution plan at runtime by replacing the scientists' predefined preferences or suggesting changes to be confirmed by scientists.

7. Concluding Remarks

The exploratory nature of the science and the increase of performance and availability of computational resources contributed to the growing of complex computational experiments. Managing the trials of these complex experiments with their respective data generated from several workflows execution is becoming an unfeasible task without an appropriate computation support. It is continuously necessary to propose new approaches to consolidate all these data and new abstraction models to abstract the experiment information in order to ease the work of the scientist. Current approaches for workflow configuration do not to support the scientist in linking the resulting workflow execution data to its abstract definition. This association is essential to avoid testing the same configuration and to reassure which result is from which workflow definition.

In this work we take advantage of the concept of provenance data to link the execution resulting data to the workflow abstract concepts. We present a practical approach to deriving workflows using experiment lines based on an algebraic model. The AEL approach is a way to ease the creation and control of the several workflows that are configured and executed during the experiment life cycle in a unique and integrated representation model. From this model, the implementation alternatives of an experiment are expressed in a high abstract language without considering technical issues. At the same time, corresponding execution results are available for the experiment data analysis. When an experiment line is defined, abstract workflows can be derived and instantiated into concrete workflows to be further executed in specific SWfMS, representing a particular, addressable, experiment alternative implementation. The AEL approach is available with the ExpLine tool, which is available as Open Source software at https://github.com/dew-uff/expline.

With the use of the workflow algebra SciWfA we improved consistency among the abstraction levels by means of a uniform data model that added semantics to the experiment line and derived abstract and concrete workflows. Associated to that, the definition of a provenance model that maps the information of the different abstraction levels allowed for validation and traceability along the workflow derivation process. Furthermore, scientists can now analyze the experiment results combining different workflow trials. This integration is facilitated by the workflow algebra definition, which correlates execution data to experiment high-level concepts defined in the experiment line model also with an algebraic approach. These characteristics associated to the expressiveness power of experiment line model to represent the experiment in terms of variability, optionality, and customized composition rules (e.g., dependencies and conflicts relationship) make the AEL approach one step ahead from the related work we found in the literature, since approaches that support higher abstract level models cannot link them to the concrete level.

The expressiveness of AEL allowed for a controlled and formalized workflow derivation process, which in the case study was able to aggregate 30 trials and their corresponding executions. The provenance model contributed by being the concretization of the mappings between concepts from the different abstraction levels. It innovates by going from the experiment line abstraction level until the execution provenance data. In fact it improved the overall analysis by exposing which of the MSA activity experiment variations, in this case the three alternative programs, produced the most efficient result. Without such provenance integration support provided by ExpLine, scientists would be fated to explore

several workflows trials individually and/or use tacit information to try to discover relationships between the experiment concepts and make some assumptions.

Moreover, in the future, the AEL derived concrete workflows can be further optimized by doing algebraic transformations. The algebraic model provides the data consumption and production semantics to the abstract workflow activities. Based on the provenance database, AEL may suggest concrete activities that usually perform faster or suggest activity variations that contain the filter operator, so that the dataflow may be reduced for performance issues. This optimization would potentially allow for a reduction of the amount of data to be processed and for the definition of parallelism points during the workflow execution. AEL could improve the optimization process to a higher level if the experiment variabilities are used on the cost evaluation of workflow execution plans. A new workflow execution plan could consider changing an approach of a workflow activity or adding/removing an optional workflow activity. All these modifications could be done at runtime, where the workflow execution engine would interact with AEL validating if all the experiment line rules are met. In the negative case, other modifications could be calculated and added to the new workflow execution plan.

Acknowledgements

This work was partially funded by Brazilian agencies FAPERJ, CNPq and CAPES.

References

[1] I. J. Taylor, E. Deelman, D. B. Gannon, and M. Shields, *Workflows for e-Science: Scientific Workflows for Grids*, 1st ed. Springer, 2007.

[2] K. Maheshwari, C. Goble, P. Missier, and J. Montagnat, "Medical image processing workflow support on the EGEE grid with taverna," in *22nd IEEE International Symposium on Computer-Based Medical Systems, 2009. CBMS 2009*, 2009, pp. 1–7.

[3] E. Santos, L. Lins, J. Ahrens, J. Freire, and C. Silva, "VisMashup: streamlining the creation of custom visualization applications," *IEEE Trans Vis Comput Graph*, vol. 15, no. 6, pp. 1539–1546, Dec. 2009.

[4] P. Fox, D. McGuinness, D. Middleton, L. Cinquini, J. A. Darnell, J. Garcia, P. West, J. Benedict, and S. Solomon, "Semantically-Enabled Large-Scale Science Data Repositories," in *The Semantic Web - ISWC 2006*, vol. 4273, I. Cruz, S. Decker, D. Allemang, C. Preist, D. Schwabe, P. Mika, M. Uschold, and L. M. Aroyo, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 792–805.

[5] W. Michener, J. Beach, S. Bowers, L. Downey, M. Jones, B. Ludäscher, D. Pennington, A. Rajasekar, S. Romanello, M. Schildhauer, D. Vieglais, and J. Zhang, "Data Integration and Workflow Solutions for Ecology," in *Data Integration in the Life Sciences*, vol. 3615, B. Ludäscher and L. Raschid, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 321–324.

[6] W. Sudholt, I. Altintas, and K. Baldridge, "Scientific Workflow Infrastructure for Computational Chemistry on the Grid," in *Computational Science – ICCS 2006*, vol. 3993, V. N. Alexandrov, G. D. Albada, P. M. A. Sloot, and J. Dongarra, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 69–76.

[7] J. Blythe, S. Jain, E. Deelman, Y. Gil, K. Vahi, A. Mandal, and K. Kennedy, "Task scheduling strategies for workflow-based applications in grids," in *Cluster Computing and the Grid, 2005. CCGrid 2005. IEEE International Symposium on*, 2005, vol. 2, pp. 759–767 Vol. 2.

[8] K. A. C. S. Ocaña, D. Oliveira, J. Dias, E. Ogasawara, and M. Mattoso, "Optimizing Phylogenetic Analysis Using SciHmm Cloud-based Scientific Workflow," in *Proceedings of the 7th IEEE International Conference on e-Science (e-Science)*, Stockholm, Sweden, 2011, pp. 190–197.

[9] J. Goecks, A. Nekrutenko, and J. Taylor, "Galaxy: a comprehensive approach for supporting accessible, reproducible, and transparent computational research in the life sciences," *Genome Biology*, vol. 11, no. 8, p. 86, 2010.

[10] T. Oinn, M. Addis, J. Ferris, D. Marvin, M. Senger, M. Greenwood, T. Carver, K. Glover, M. R. Pocock, A. Wipat, and P. Li, "Taverna: a tool for the composition and enactment of bioinformatics workflows," *Bioinformatics*, vol. 20, no. 17, pp. 3045–3054, 2004.

[11] J. Dias, G. Guerra, F. Rochinha, A. L. G. A. Coutinho, P. Valduriez, and M. Mattoso, "Data-centric iteration in dynamic workflows," *Future Generation Computer Systems*, vol. 46, pp. 114–126, May 2015.

[12] R. Stevens, K. Glover, C. Greenhalgh, C. Jennings, S. Pearce, P. Li, M. Radenkovic, A. Wipat, C. Tower, and J. Campus, "Performing in silico experiments on the Grid: a users perspective," in *Proceedings of the UK e-Science programme All Hands Conference*, Nottingham, UK, 2003, pp. 2–4.

[13] A. Goderis, U. Sattler, P. Lord, and C. Goble, "Seven Bottlenecks to Workflow Reuse and Repurposing," in *The Semantic Web – ISWC 2005*, Galway, Ireland, 2005, pp. 323–337.

[14] J. Sacks, W. J. Welch, T. J. Mitchell, and H. P. Wynn, "Design and Analysis of Computer Experiments," *Statist. Sci.*, vol. 4, no. 4, pp. 409–423, Nov. 1989.

[15] M. Mattoso, C. Werner, G. H. Travassos, V. Braganholo, E. Ogasawara, D. Oliveira, S. Cruz, W. Martinho, and L. Murta, “Towards supporting the life cycle of large scale scientific experiments,” *International Journal of Business Process Integration and Management*, vol. 5, no. 1, pp. 79–92, 2010.
[16] S. P. Callahan, J. Freire, E. Santos, C. E. Scheidegger, C. T. Silva, and H. T. Vo, “VisTrails: Visualization Meets Data Management,” in *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*, New York, NY, USA, 2006, pp. 745–747.
[17] D. Oliveira, E. Ogasawara, F. Baião, and M. Mattoso, “SciCumulus: a lightweight cloud middleware to explore many task computing paradigm in scientific workflows,” in *3rd International Conference on Cloud Computing*, Washington, DC, USA, 2010, pp. 378–385.
[18] I. Altintas, C. Berkley, E. Jaeger, and M. Jones, “Kepler: An Extensible System for Design and Execution of Scientific Workflows.” 2004.
[19] D. Hull, K. Wolstencroft, R. Stevens, C. Goble, M. R. Pocock, P. Li, and T. Oinn, “Taverna: a tool for building and running workflows of services,” *Nucleic Acids Research*, vol. 34, no. 2, pp. 729–732, 2006.
[20] M. Wilde, M. Hategan, J. M. Wozniak, B. Clifford, D. S. Katz, and I. Foster, “Swift: A language for distributed parallel scripting,” *Parallel Computing*, no. 37(9), pp. 633–652, 2011.
[21] E. Deelman, G. Mehta, G. Singh, M.-H. Su, and K. Vahi, “Pegasus: Mapping Large-Scale Workflows to Distributed Resources,” in *Workflows for e-Science*, I. J. Taylor, E. Deelman, D. B. Gannon, and M. Shields, Eds. Springer, 2007, pp. 376–394.
[22] T. Fahringer, R. Prodan, Rubing Duan, F. Nerieri, S. Podlipnig, Jun Qin, M. Siddiqui, Hong-Linh Truong, A. Villazon, and M. Wieczorek, “ASKALON: a Grid application development and computing environment,” in *6th IEEE/ACM International Workshop on Grid Computing*, Seattle, Washington, USA, 2005, pp. 122–131.
[23] A. Shoshani and D. Rotem, *Scientific Data Management: Challenges, Technology, and Deployment*, 1st ed. Chapman and Hall/CRC, 2009.
[24] L. Salayandia, P. P. da Silva, A. Q. Gates, and F. Salcedo, “Workflow-Driven Ontologies: An Earth Sciences Case Study,” in *Proceedings of the Second IEEE International Conference on e-Science and Grid Computing*, 2006, p. 17.
[25] B. Ludascher, I. Altintas, and A. Gupta, “Compiling abstract scientific workflows into web service workflows,” in *Scientific and Statistical Database Management*, Cambridge, MA, 2003, pp. 251–254.
[26] C. Lin, S. Lu, X. Fei, D. Pai, and J. Hua, “A Task Abstraction and Mapping Approach to the Shimming Problem in Scientific Workflows,” in *Proc. Services 2009*, Bangalore, India, 2009, pp. 284–291.
[27] J. Qin, T. Fahringer, and S. Pllana, “UML based Grid Workflow Modeling under ASKALON,” in *Distributed and Parallel Systems*, 2007, pp. 191–200.
[28] M. Acher, P. Collet, P. Lahire, and R. France, “Managing Variability in Workflow with Feature Model Composition Operators,” in *Software Composition*, vol. 6144, B. Baudry and E. Wohlstadter, Eds. Springer Berlin / Heidelberg, 2010, pp. 17–33.
[29] K. Plankensteiner, J. Montagnat, and R. Prodan, “IWIR: a language enabling portability across grid workflow systems,” in *Proceedings of the 6th workshop on Workflows in support of large-scale science*, New York, NY, USA, 2011, pp. 97–106.
[30] K. Belhajjame, D. Garijo, G. Klyne, R. Palma, P. Holubowicz, and E. Garcia-Cuesta, “D2.2v2 Design, implementation and deployment of workflow lifecycle management components - Phase II,” http://www.wf4ever-project.org/, 2014.
[31] M. D. Wilkinson, B. Vandervalk, and L. McCarthy, “The Semantic Automated Discovery and Integration (SADI) Web service Design-Pattern, API and Reference Implementation,” *Journal of Biomedical Semantics*, vol. 2, no. 1, p. 8, Oct. 2011.
[32] Y. Gil, V. Ratnakar, E. Deelman, G. Mehta, and J. Kim, “Wings for Pegasus: Creating Large-Scale Scientific Applications Using Semantic Representations of Computational Workflows,” in *The National Conference On Artificial Intelligence*, Vancouver, BC, Canada, 2007, vol. 22, pp. 1767–1774.
[33] J. Freire, D. Koop, E. Santos, and C. T. Silva, “Provenance for Computational Tasks: A Survey,” *Computing in Science Engineering*, vol. 10, no. 3, pp. 11–21, 2008.
[34] E. Ogasawara, C. Paulino, L. Murta, C. Werner, and M. Mattoso, “Experiment Line: Software Reuse in Scientific Workflows,” in *Scientific and Statistical Database Management*, New Orleans, Louisiana, USA, 2009, pp. 264–272.
[35] E. Ogasawara, J. Dias, D. Oliveira, F. Porto, P. Valduriez, and M. Mattoso, “An Algebraic Approach for Data-Centric Scientific Workflows,” *Proceedings of the 37th International Conference on Very Large Data Bases (PVLDB)*, vol. 4, no. 12, pp. 1328–1339, 2011.
[36] E. F. Codd, *The relational model for database management: version 2*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1990.
[37] J. Dean and S. Ghemawat, “MapReduce: a flexible data processing tool,” *Communications of the ACM*, vol. 53, no. 1, pp. 72–77, 2010.
[38] J. C. Jacob, D. S. Katz, G. B. Berriman, J. C. Good, A. C. Laity, E. Deelman, C. Kesselman, G. Singh, M.-H. Su, T. A. Prince, and R. Williams, “Montage: a grid portal and software toolkit for science-grade astronomical image mosaicking,” *International Journal of Computational Science and Engineering (IJCSE)*, vol. 4, no. 2, pp. 73–87, 2009.
[39] V. Silva, D. Oliveira, and M. Mattoso, “Exploratory analysis of raw data files through dataflows,” in *Proceedings of the Workshop on Parallel and Distributed Computing for Big Data Applications (WPBA 2014)*, Paris, France, 2014, pp. 114–119.

[40] E. W. Greisen and M. R. Calabretta, “Representations of world coordinates in FITS,” *Astronomy and Astrophysics*, vol. 395, no. 3, pp. 1061–1075, Dec. 2002.
[41] D. Makovoz and I. Khan, “Mosaicking with MOPEX,” presented at the Proceedings of ADASS XIV, Pasadena, 2004.
[42] F. G. Valdes, “The IRAF Mosaic Data Reduction Package,” *Astronomical Data Analysis Software and Systems VII*, vol. 145, 1998.
[43] R. N. Elias and A. L. G. A. Coutinho, “Stabilized edge-based finite element simulation of free-surface flows,” *International Journal for Numerical Methods in Fluids*, vol. 54, no. 6–8, pp. 965–993, 2007.
[44] G. Guerra, F. A. Rochinha, R. Elias, D. de Oliveira, E. Ogasawara, J. F. Dias, M. Mattoso, and A. L. G. A. Coutinho, “Uncertainty Quantification in Computational Predictive Models for Fluid Dynamics Using Workflow Management Engine,” *International Journal for Uncertainty Quantification*, vol. 2, no. 1, pp. 53–71, 2012.
[45] E. F. Lins, R. N. Elias, G. M. Guerra, F. A. Rochinha, and A. L. G. A. Coutinho, “Edge-based finite element implementation of the residual-based variational multiscale method,” *International Journal for Numerical Methods in Fluids*, vol. 61, no. 1, pp. 1–22, 2009.
[46] I. Taylor, M. Shields, I. Wang, and A. Harrison, “The Triana Workflow Environment: Architecture and Applications,” in *Workflows for e-Science*, Springer, 2007, pp. 320–339.
[47] D. Oliveira, E. Ogasawara, F. Seabra, V. Silva, L. Murta, and M. Mattoso, “GExpLine: A Tool for Supporting Experiment Composition,” in *Provenance and Annotation of Data and Processes*, 2010, pp. 251–259.
[48] L. M. Northrop, “SEI’s software product line tenets,” *IEEE Software*, vol. 19, no. 4, pp. 32–40, 2002.
[49] I. Bratko, *Prolog programming for artificial intelligence*. Harlow, England; New York: Addison Wesley, 2001.
[50] E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
[51] A. Marinho, L. Murta, C. Werner, V. Braganholo, S. M. S. da Cruz, E. Ogasawara, and M. Mattoso, “ProvManager: a provenance management system for scientific workflows,” *Concurrency and Computation: Practice and Experience*, vol. 24, no. 13, pp. 1513–1530, 2012.
[52] F. Costa, V. Silva, D. de Oliveira, K. Ocaña, E. Ogasawara, J. Dias, and M. Mattoso, “Capturing and querying workflow runtime provenance with PROV: a practical approach,” in *Proceedings of the Joint EDBT/ICDT 2013 Workshops*, New York, NY, USA, 2013, pp. 282–289.
[53] L. Moreau, P. Missier, and B. Belhajjame, *The PROV Data Model and Abstract Syntax Notation*. 2013.
[54] Amazon EC2, “Amazon Elastic Compute Cloud (Amazon EC2),” http://aws.amazon.com/ec2/, 2010.
[55] Hadoop, “Apache Hadoop Web page,” http://hadoop.apache.org/, 2012.
[56] D. de Oliveira, K. A. C. S. Ocaña, F. Baião, and M. Mattoso, “A Provenance-based Adaptive Scheduling Heuristic for Parallel Scientific Workflows in Clouds,” *J Grid Computing*, vol. 10, no. 3, pp. 521–552, 2012.
[57] J. W. Leigh, F.-J. Lapointe, P. Lopez, and E. Bapteste, “Evaluating Phylogenetic Congruence in the Post-Genomic Era,” *Genome Biology and Evolution*, vol. 3, no. 0, pp. 571–587, Jun. 2011.
[58] D. Oliveira, K. A. C. S. Ocaña, E. Ogasawara, J. Dias, J. Gonçalves, F. Baião, and M. Mattoso, “Performance evaluation of parallel strategies in public clouds: A study with phylogenomic workflows,” *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1816–1825, Sep. 2013.
[59] D. Oliveira, K. A. C. S. Ocaña, E. Ogasawara, J. Dias, J. Goncalves, and M. Mattoso, “Cloud-based Phylogenomic Inference of Evolutionary Relationships: A Performance Study,” in *Proceedings of the 2nd International Workshop on Cloud Computing and Scientific Applications (CCSA)*, Ottawa, Canadá, 2012.
[60] S. Wu and U. Manber, “Fast text searching: allowing errors,” *Communications of the ACM*, vol. 35, no. 10, pp. 83–91, Oct. 1992.
[61] J. Felsenstein, “PHYLIP - Phylogeny Inference Package (Version 3.2),” *Cladistics*, vol. 5, pp. 164–166, 1989.
[62] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 6th ed. Addison-Wesley, 2010.